FPGA Tinkering Plan

Personal project + Endura bench correlation — Marko Mijatovic

Goal: Build a working FPGA digital design (satisfies the HDL / vendor-tool / project requirement for FPGA-ASIC hardware roles) that also produces real hardware-correlation data for the Endura PDN tool, which currently has that gap scoped out.

1. The Project: Transient Capture & Trigger Core

An FPGA-based digitizer that watches a real PCB load-step, triggers on the voltage droop, buffers the waveform, and streams it out for comparison against the PDN nodal solver's prediction.

Core blocks to design

1. **ADC interface FSM** — SPI (or onboard ADC) read state machine, fixed sample rate.
2. **Trigger-detect logic** — comparator against a programmable threshold register; detects the droop edge.
3. **Circular buffer (Block RAM)** — captures pre-trigger and post-trigger samples continuously; freezes on trigger.
4. **UART TX core** — streams the captured buffer to a PC for logging / plotting.
5. **Top-level integration** — wire the above into one design, add a self-checking testbench for each block.

Build order (do this in sequence, don't skip)

1. Blink an LED on the dev board. Confirms toolchain, constraints file, and bitstream flow all work.
2. Write and simulate the UART TX core alone (easiest block, most tutorials exist). Verify in simulation before touching hardware.
3. Write the circular buffer + trigger logic, simulate with a fake sawtooth/step waveform as stimulus.
4. Wire in the real ADC over SPI. Debug on hardware with an actual signal (function generator or the PCB itself).
5. Integrate all blocks; capture a real load-step droop; export data; compare against your PDN solver's prediction.
6. Write it up: one page, block diagram, waveform screenshot, solver-vs-hardware comparison plot.

Why this project (not something generic)

- It is a real HDL design with FSMs, memory, and a testbench — exactly what satisfies "built a working FPGA project" on job requirements.
- It directly produces the hardware-correlation data your Endura tool is missing, so it's not just a resume line — it's real infra you can hand to your team.
- It's scoped small enough to finish in a few weekends, unlike a full SoC or CPU.

2. Software to Download

1. **Xilinx Vivado (free WebPACK edition)** — `https://www.xilinx.com/support/download.html`. This is the vendor tool (synthesis, place-and-route, bitstream generation, simulation).
2. **A Verilog/SystemVerilog simulator for fast iteration** — Vivado's built-in simulator works, but **Icarus Verilog** (`iverilog`) + **GTKWave** is faster to iterate with and free/open-source. Install via your package manager or `http://iverilog.icarus.com/`.
3. **cocotb** (Python-based testbenches) — you already list this on your resume; use it here for real. `pip install cocotb`. Pairs with Icarus Verilog as the simulation backend.
4. **A serial terminal** for UART debug — `minicom`, `screen`, or PuTTY.

3. Hardware to Buy

- **Digilent Arty A7-35T** (~$130) — Xilinx Artix-7, PMOD headers for the ADC module, large community/tutorial base. Recommended default.
- Alternative if budget-constrained: **Digilent Basys3** (~$150, also Artix-7, no onboard ADC precision needed if you use an external PMOD ADC).
- **PMOD ADC module** (e.g., Digilent PMOD AD1 or AD2) to read the real voltage droop signal.

4. What to Watch / Read (in order)

1. **Nandland "Verilog in 10 Days" style intro series** (YouTube, search "Nandland Verilog basics") — fastest on-ramp to Verilog syntax and simulation mindset if you haven't written RTL before.
2. **Digilent's official Vivado + Arty A7 getting-started tutorial** (search "Digilent Arty A7 Vivado getting started") — walks the LED-blink-to-bitstream flow end to end, matches the board you're buying.
3. **cocotb official documentation quickstart** (`https://docs.cocotb.org`) — read the "Quickstart" page directly; short and you already know Python.
4. **Ben Eater's digital logic / breadboard CPU series** (YouTube) — not FPGA-specific, but the best intuition-builder for how FSMs and memory actually behave in hardware if any of that feels shaky.
5. **Search "UART Verilog tutorial testbench"** — multiple short walkthroughs exist for exactly the block you're building first; use one as a reference, don't copy line-for-line (know what every line does).

5. Milestones / Rough Timeline

- **Week 1:** Vivado + board setup, LED blink working, Icarus + GTKWave installed and simulating a trivial module.
- **Week 2:** UART TX core designed, simulated, and verified on real hardware (loop bytes to your laptop terminal).
- **Week 3:** Trigger logic + circular buffer designed and simulated against a fake waveform.
- **Week 4:** ADC integration, full system wired together, first real capture.
- **Week 5:** Compare captured droop against PDN solver output, write up the one-pager, update resume with the real bullet.

Note: don't add this project or Vivado/Verilog to your resume until there's a working capture and a testbench you can defend in an interview. Padding this now is worse than leaving it blank.